🎓 SkillX Project Viva Questions

This document contains a curated list of potential viva questions asked by external examiners, industry experts, or professors regarding your **SkillX** project.

◇ 1. General Project & Problem Statement

Q1: What is the core problem your project "SkillX" solves?

> **Answer:** It solves the problem of expensive or unstructured learning. Instead of paying for courses or aimlessly searching on social media, SkillX connects specific skill providers with skill seekers in a structured, peer-to-peer barter system.

Q2: What is the "Unique Selling Point" (USP) of your project?

> **Answer:** The **Shared Session Passkey** mechanism. Unlike standard meeting links that can be shared unauthorizedly, our system generates a unique, temporary 5-character passkey that is only valid for the specific session participants, ensuring security.

Q3: Why did you choose the MERN stack for this project?

> **Answer:**
>
> - **JavaScript everywhere:** Single language for both frontend and backend.
> - **JSON:** Native support for JSON in MongoDB and Node.js makes data flow seamless.
> - **React:** Component-based architecture allows for a dynamic and responsive UI.
> - **Node.js:** Non-blocking I/O is excellent for real-time features like our chat system.

Q4: How does this differ from LinkedIn or standard tutoring sites?

> **Answer:** LinkedIn is for professional networking, not necessarily immediate skill exchange sessions. Tutoring sites often require money. SkillX is based on a **barter economy** (Skill-for-Skill), making knowledge accessible to everyone regardless of financial status.

◇ 2. Backend & Database (Node.js, Express, MongoDB)

Q5: Can you explain your Database Schema?

> **Answer:** We use **MongoDB** (NoSQL). Key collections likely include:
>
> - **Users:** Stores specific details, `skillsOffered` array, `skillsNeeded` array, and hashed passwords.
> - **Requests:** Tracks connection status (Pending, Accepted, Rejected).
> - **Sessions:** Stores active session IDs, passkeys, and logs.

Q6: Why NoSQL (MongoDB) instead of SQL (MySQL)?

> **Answer:**

> - **Flexibility:** User profiles and skill lists can vary in structure.
> - **Scalability:** Horizontal scaling is easier with MongoDB.
> - **Rapid Development:** No need for complex joins for simple data retrieval; documents map directly to objects in our code.

Q7: How are you handling Passwords?

> **Answer:** We **never** store plain text passwords. We use **bcryptjs** to hash passwords (adding salt) before saving them to the database. During login, we compare the entered password with the stored hash.

Q8: Explain the logic behind the "Shared Session Passkey".

> **Answer:** When a request is accepted, the backend generates a random 5-character alphanumeric string. This string is stored in the database linked to that specific session ID. When a user tries to join, the backend verifies:
>
> 1. Does the passkey exist?
> 2. Does the user ID belong to one of the participants of that session?

◇ 3. Frontend & React.js

Q9: Why did you use React.js?

> **Answer:** Its **Virtual DOM** ensures high performance by only updating changed parts of the UI. The **Component-based structure** makes the code reusable (e.g., we reuse the Navbar across pages).

Q10: How do you manage Global State (e.g., User Login Status)?

> **Answer:** We use the **React Context API** (specifically `AuthContext`) to store the logged-in user's data and token globally, so we don't have to pass props down manually through every component level ("Prop Drilling").

Q11: What is the role of `useEffect` in your code?

> **Answer:** We use `useEffect` for side effects, such as:
>
> - Fetching user data from the API when the component mounts.
> - Setting up the Socket.io connection when entering a chat room.

◇ 4. API & Authentication

Q12: What is JWT and how does it work in SkillX?

> **Answer: JSON Web Token (JWT)** is used for stateless authentication.
>
> 1. **Login:** Server verifies credentials and sends a signed JWT to the client.
> 2. **Storage:** Client stores it (usually in localStorage).
> 3. **Access:** For every protected request (like "Accept Request"), the client sends this token in the header. The server verifies the signature to know who the user is.

Q13: What is the difference between Authentication and Authorization?

> **Answer:**
>
> - **Authentication:** Verifying *who* you are (Login).
> - **Authorization:** Verifying *what* you are allowed to do (e.g., Only the session participants are *authorized* to enter the room using the passkey).

◇ 5. Real-Time Features (Socket.io)

Q14: How does the Chat feature work?

> **Answer:** We use **Socket.io**.
>
> - It uses **WebSockets** for a persistent, bi-directional connection.
> - When User A types a message, the client emits an event to the server.
> - The server broadcasts that message specifically to the room (Session ID) so User B sees it instantly without refreshing the page.

Q15: What happens if the internet disconnects during a session?

> **Answer:** Socket.io has built-in reconnection logic. It will try to reconnect automatically. However, for a production app, we would implement better "offline" indicators in the UI.

◇ 6. Future Scope & Challenges

Q16: If 1 million users join, how will your search handle it?

> **Answer:** Currently, we filter arrays in MongoDB. For 1 million users, this would be slow. We would need to implement **Indexing** on the skills fields or use a dedicated search engine like **Elasticsearch**.

Q17: How would you add Video Calling?

> **Answer:** We would use **WebRTC (Web Real-Time Communication)**. We would use our existing Socket.io connection to exchange "Signaling" data (SDP and ICE candidates) to establish a direct peer-to-peer video stream between browsers.

Q18: What was the most challenging part of this project?

> **Answer:** (Personalize this, but a good standard answer is):
>
> - *"Managing the state for the live session. ensuring that when one user accepts a request, the session is created instantly and both users get the correct passkey."*
> - *OR "Handling the asynchronous nature of Node.js when querying the database and waiting for external services."*